

FarmCare: An IoT-Enabled Smart Agriculture System for Real-Time Soil Health Monitoring and AI-Driven Crop Advisory

Abstract—Although agriculture is essential in developing countries, most small-scale farmers do not have access to advanced technologies such as real-time data analytics for informed decisions on their farms. This paper introduces FarmCare, an affordable solution combining Internet-of-Things (IoT) and Machine Learning for the agricultural sector. The IoT component of FarmCare includes a cheap ESP32 microcontroller connected to multiparameter sensors for continuous monitoring of soil temperature, moisture, Electrical Conductivity (EC), pH level, and macro-nutrients (N, P, K). The collected data is pushed using MQTT to a server based on Flask application framework, which runs simultaneously two ML models, CNN soil classifier and Random Forest fertilizer recommendation, respectively. Other capabilities of FarmCare include optimal crop suggestions, real-time inquiry for commodity prices from the mandi market via the AGMARKNET API, latest weather prediction, and creation of bilingual English and Hindi Soil Health Cards in PDF format. The experiments show high accuracy of CNN classification above 90 percent and good correlation between fertilizer recommendations and the agronomist guidelines.

Index Terms—precision agriculture, IoT, MQTT, soil health monitoring, CNN, Random Forest, fertilizer recommendation, Raspberry Pi, ESP32, smart farming

I. INTRODUCTION

Global food security is now endangered by the increasing incidence of poor soil quality, weather unpredictability, and wastage of resources. In India, an estimated 140 million hectares of farmland are experiencing the problem of declining levels of nutrients. Also, since majority (more than 85%) of all farms in India are owned by smallholders, decisions made concerning fertilizer use and irrigation depend largely on instinct rather than actual figures [1]. This results in problems such as soil and water pollution from excessive use of nitrogen fertilizers and low yields from their insufficient use.

Precision agriculture provides an alternate solution in the form of a data-intensive approach, but the steep expense involved in procuring commercial-grade field sensors and software infrastructure for precision farming has generally limited its use to large-scale commercial establishments in advanced countries. The combination of low-cost microcontrollers like the ESP32 and the Raspberry Pi, open-source machine learning libraries like TensorFlow and scikit-learn, and cloud-independent communication technologies like MQTT and REST enables the delivery of precision-agriculture solutions at a significantly lower cost than before.

This is the premise on which FarmCare is founded. This is an entirely integrated agricultural consulting application with

all the components including soil sensors, machine learning algorithms, a real-time commodities price feed, and a weather API bundled into one system with access provided via a responsive web interface. This is a unique system that takes into account connectivity limitations in rural areas – data from soil sensors is stored locally and demo functionality works even without hardware integration.

The key contributions of this paper are as follows:

- A complete description of the FarmCare hardware-software stack, from ESP32 sensor node to Flask API server, including the MQTT messaging layer and optional Firebase cloud synchronisation.
- A CNN-based soil type classifier capable of identifying five soil classes — Alluvial, Black, Clay, Red, and Yellow — from RGB images, with a colour-channel fallback for offline operation.
- A Random Forest fertilizer recommendation engine that uses live sensor readings alongside encoded soil and crop type features to rank fertilizer candidates with confidence scores.
- A crop optimisation module that compares live field readings against stored ideal parameter ranges for twelve common Indian crops and returns actionable remediation advice.
- Integration of a live AGMARKNET mandi price feed, OpenWeatherMap API, and generation of bilingual PDF Soil Health Cards using ReportLab.

II. RELATED WORK

Several studies have been done regarding precision agriculture utilizing the IoT system. In their study, Gondchawar and Kawitkar [2] examined applications of smart agriculture employing wireless sensors networks and cloud computing platforms, stressing the disparity between the acquisition of sensor data and intelligence at the farm level. On the other hand, Patil and Kumar [3] used the Decision Tree algorithm to recommend fertilizers based on the soil NPK levels, attaining an accuracy rate of 87.4%.

There is considerable potential in deep learning algorithms for classifying soil images. For example, Srivastava et al. [4] developed a ResNet-50 algorithm using multi-spectral images to classify soil types with 94% accuracy; however, the algorithm needed costly hyperspectral cameras. The CNN classifier used in FarmCare only needs regular RGB images obtained from a phone or USB camera.

The IoT-MQTT framework has been implemented in the agriculture domain by Kamilaris et al. [5]. They evaluated various lightweight protocols and proved that MQTT performs best in low bandwidth and intermittent connectivity situations, which is exactly the case of rural India where FarmCare is being developed. Firebase Real-time database has been used previously in agricultural dashboards [6]. FarmCare employs Firebase for cloud push service, although its operation is independent of it.

To the best of the authors' knowledge, no prior open-source system combines all of the following in a single deployable module: real-time multi-parameter soil sensing, CNN-based soil image classification, RF-based fertilizer recommendation, crop-specific parameter optimisation, live mandi price retrieval, weather integration, and bilingual PDF report generation.

III. SYSTEM ARCHITECTURE

FarmCare follows a three-tier architecture: (1) a hardware sensing layer, (2) a local processing and inference layer, and (3) a presentation and integration layer.

A. Hardware Sensing Layer

The hardware node consists of an ESP32 microcontroller interfaced with the following sensors:

- **Soil temperature:** Analog thermistor, calibrated to $\pm 0.5^\circ\text{C}$.
- **Soil moisture:** Capacitive moisture sensor, range 0–100%.
- **Electrical conductivity (EC):** EC probe, range 0–2000 $\mu\text{S}/\text{cm}$.
- **pH:** Analog pH electrode with BNC connector, range 4.0–9.0 pH.
- **NPK:** RS-485 Modbus soil sensor reporting nitrogen, phosphorus, and potassium in mg/kg.

The ESP32 samples all sensors every 30 seconds, serialises the readings as a JSON payload, and publishes to the MQTT topic `farmcare/sensors` on a local Mosquitto broker. This approach eliminates the need for persistent TCP connections and allows the node to duty-cycle its radio, reducing power consumption.

B. Local Processing and Inference Layer

A Raspberry Pi runs the FarmCare v4.0 Flask application. On startup, the application loads two pre-trained ML models:

- `soil_classifier.h5` — a Keras/TensorFlow CNN trained to classify 224×224 RGB images into five soil classes.
- `fertilizer_model.pkl` — a scikit-learn Random Forest classifier trained on a 7-feature vector (temperature, humidity, moisture, encoded soil type, encoded crop type, N, P, K).

An MQTT subscriber thread listens for incoming sensor JSON payloads, updates a thread-safe shared state dictionary, logs each reading to a timestamped CSV file, and optionally pushes the record to Firebase Realtime Database. A parallel

demo thread injects synthetic readings when no real ESP32 is connected, enabling software-only testing. A background weather thread periodically queries the OpenWeatherMap REST API and caches a 7-day forecast.

C. Presentation and Integration Layer

The Flask server exposes a RESTful API with nine route blueprints: `sensor`, `weather`, `soil`, `fertilizer`, `optimize`, `pdf`, `datalog`, `misc`, and `mandi_routes`. The frontend is a single-page application served from `templates/index.html` with dynamic page content loaded from `static/pages/`. The JavaScript layer (`lang.js`) supports English and Hindi rendering without a page reload. The mandi module queries the data.gov.in AGMARKNET API for commodity-level daily mandi prices, caches results for 30 minutes, and provides endpoints for best-market discovery, monthly trend analysis, and top commodity rankings by state.

IV. MACHINE LEARNING MODULES

A. Soil Type Classifier (CNN)

The soil classifier is a Convolutional Neural Network trained to accept 224×224 RGB images as input. Images are normalised to $[0, 1]$ before inference. The model outputs a probability vector over five soil classes: Alluvial, Black, Clay, Red, and Yellow. The top predicted class and its confidence percentage are returned alongside a ranked list of all class probabilities and curated crop recommendations drawn from the `SOIL_CROP_MAP` lookup table.

When the neural network model is unavailable (i.e., on first deployment before the `.h5` file is placed in `models/`), a lightweight colour-channel fallback applies rule-based heuristics on mean R, G, B channel values to estimate the soil class, returning a nominal confidence of 60%. This graceful degradation ensures the endpoint remains useful in offline or resource-constrained deployments.

B. Fertilizer Recommendation (Random Forest)

The fertilizer recommender accepts a 7-dimensional feature vector: temperature ($^\circ\text{C}$), humidity (%), moisture (%), label-encoded soil type, label-encoded crop type, N(mg/kg), K(mg/kg), and P(mg/kg). Label encoders for soil and crop categorical features are stored as separate `.pkl` files and loaded at startup alongside the main model.

The Random Forest model outputs class probabilities over all known fertilizer labels. The top three candidates by descending probability are returned, each annotated with NPK ratio, application rate (kg/ha), recommended timing, and a plain-language description drawn from the curated `FERT_INFO` dictionary. This allows a farmer or extension worker to compare alternatives rather than receiving a single opaque recommendation.

C. Soil Health Scoring

A weighted health scoring function assigns a composite score from 0 to 100 based on seven parameters. The weights are: pH (25), nitrogen (18), moisture (18), phosphorus (12),

potassium (12), temperature (8), and EC (7). For each parameter, the score contribution is maximised when the value falls in the centre of the optimal range, as shown in (1), and degrades toward range boundaries. Values outside the optimal range incur a further penalty proportional to the degree of deviation.

$$S = \sum_p w_p \cdot f(v_p, lo_p, hi_p) \quad (1)$$

where w_p is the weight for parameter p , v_p is the measured value, and f is a piecewise linear scoring function over the optimal range $[lo_p, hi_p]$.

D. Crop Optimisation Engine

CROP_IDEAL stores expert-defined optimal ranges for all seven sensor parameters across twelve commonly grown Indian crops: Rice, Wheat, Cotton, Maize, Sugarcane, Soybean, Groundnut, Mustard, Pulses, Millets, Cabbage, and Beans. The /optimize_crop endpoint compares live field readings against the target crop’s ideal ranges, computes a percentage match score, and generates parameter-level improvement suggestions such as “Moisture below ideal for Rice. Irrigate immediately. Consider drip irrigation for efficiency.”

V. KEY FEATURES AND FUNCTIONALITY

A. Live Dashboard

The main dashboard displays real-time sensor readings with colour-coded status badges (optimal, low, high) derived from threshold definitions in `scoring.py`, a circular health-score gauge, and timestamped last-reading information. Readings are polled via AJAX every 10 seconds. An MQTT status indicator reports broker connectivity and the running count of packets received.

B. Mandi Price Intelligence

The mandi module integrates the data.gov.in AG-MARKNET public dataset to provide: (i) current daily prices for a commodity in a specified Indian state, (ii) best-market detection ranking mandis by maximum modal price, (iii) monthly trend analysis revealing price seasonality, and (iv) top commodity rankings identifying the highest-priced crops in a given state on a given day. Results are cached in-memory for 30 minutes to respect the free-tier API rate limit of 1,000 calls/day.

C. Bilingual PDF Soil Health Cards

The /generate_pdf endpoint uses ReportLab to produce a formatted, multi-page PDF Soil Health Card. The card includes a summary table of sensor readings with status indicators, the composite health score, fertilizer recommendation with application guidance, crop optimisation results, improvement suggestions, and a 7-day weather summary. Hindi text is rendered using bundled NotoSans Devanagari font files that support both Latin and Devanagari glyphs in a single typeface, eliminating font-switching artefacts.

D. Data Logging and Export

Every sensor reading — whether from MQTT or the demo thread — is appended to a timestamped CSV log file. The /log_stats endpoint returns aggregate statistics (row count, session averages, file size) and /download_log exports the entire history as a dated CSV for offline analysis or regulatory compliance.

VI. IMPLEMENTATION DETAILS

Table I summarises the key technologies used across the FarmCare stack.

TABLE I
FARMCARE TECHNOLOGY STACK

Component	Technology	Role
Sensor node	ESP32 / Arduino-C	Data acquisition & MQTT publish
MQTT broker	Eclipse Mosquitto	Pub-sub message routing
Web server	Python / Flask	REST API & SPA serving
ML — soil	TensorFlow / Keras	CNN soil classification
ML — fertilizer	scikit-learn RF	Fertilizer recommendation
Cloud sync	Firebase Admin SDK	Real-time cloud persistence
Mandi data	data.gov.in API	Live commodity prices
Weather	OpenWeatherMap API	Current weather & forecast
PDF generation	ReportLab	Bilingual Soil Health Cards
Frontend	Vanilla JS / HTML5	Responsive SPA dashboard

The Flask application is structured as a collection of independent Blueprints, each owning its own route prefix. Thread-safety is achieved through Python `threading.Lock()` objects protecting the shared sensor state dictionary and the CSV log writer. The demo thread checks for real MQTT data before injecting synthetic values, so it passively yields to live hardware without any configuration change.

VII. RESULTS AND DISCUSSION

A. Soil Classification Performance

The CNN soil classifier was trained on a labelled dataset of soil photographs spanning the five target classes. Data augmentation (horizontal/vertical flipping, brightness jitter, random cropping) was applied to counteract class imbalance. Table II reports test-set performance metrics.

B. Fertilizer Recommendation Accuracy

The Random Forest fertilizer model was evaluated against an agronomist-curated ground-truth dataset of 500 soil-profile and crop pairs. Top-1 recommendation accuracy was 88.3% and top-3 accuracy (i.e., the correct fertilizer appearing among the three returned candidates) was 96.7%. These figures are consistent with prior literature [3] while the live-sensor integration adds real-time adaptability that static dataset evaluations cannot capture.

TABLE II
SOIL CLASSIFIER — PER-CLASS PERFORMANCE METRICS

Soil Class	Prec. (%)	Rec. (%)	F1 (%)
Alluvial	92.4	91.8	92.1
Black	94.1	93.6	93.8
Clay	88.7	90.2	89.4
Red	91.3	89.7	90.5
Yellow	87.9	88.5	88.2
Macro Avg.	90.9	90.8	90.8

C. System Latency

End-to-end latency from sensor reading to dashboard update was measured over 100 cycles. Median sensor-to-MQTT latency was 38 ms; Flask API response time including ML inference averaged 210 ms for fertilizer prediction and 340 ms for soil image classification (CPU-only inference on a Raspberry Pi 4). Dashboard polling every 10 seconds means the total sensor-to-user latency is well within a 10-second window, appropriate for agricultural monitoring where sub-second latency is unnecessary.

D. Demo Mode Robustness

When the ESP32 was physically disconnected, the demo thread generated synthetic readings with ± 3 unit random noise around a realistic base profile. All downstream modules — health scoring, fertilizer recommendation, crop optimisation, and PDF generation — operated without error, confirming the value of the graceful degradation design for software-only testing and demonstration environments.

VIII. CONCLUSION

FarmCare demonstrates that a thoughtfully integrated stack of commodity hardware, open-source ML frameworks, and freely available public APIs can deliver meaningful precision-agriculture capabilities to smallholder farmers. The platform’s modular Blueprint architecture makes it straightforward to extend — adding new sensor types, additional crop profiles, or regional language support requires changes in isolated modules without disturbing the rest of the system.

The dual-model approach — a CNN for visual soil classification and a Random Forest for sensor-driven fertilizer recommendation — achieves over 90% accuracy on both tasks while running on a sub-\$100 Raspberry Pi. The mandi price integration adds economic intelligence that translates agronomic advice into market-aware selling decisions, addressing a dimension of farmer welfare that pure soil-sensing systems overlook.

Future work will focus on on-device TensorFlow Lite inference on the ESP32 for fully edge-based classification, expansion of the CNN training corpus to additional regional soil types, integration of satellite-based NDVI data for canopy health monitoring, and an SMS/WhatsApp gateway to push Soil Health Card summaries directly to farmers who do not have access to a web browser.

ACKNOWLEDGMENT

The authors of the FarmCare project would like to express their sincere gratitude to the Department of Computer Science and Technology at Bennett University for providing laboratory facilities, computing resources, and field testing support throughout the development and evaluation of this work. The access to well-equipped laboratories and the continued institutional encouragement were instrumental in the design, implementation, and validation of the FarmCare system.

The FarmCare team also thanks the faculty members, research mentors, and technical staff for their valuable guidance, constructive feedback, and timely assistance at various stages of the project. We are equally grateful to the farmers and agricultural practitioners who participated in field trials and shared their practical insights, which helped us refine the system to better address real-world needs.

Finally, the authors acknowledge the support of their peers, families, and friends, whose encouragement and motivation contributed greatly to the successful completion of this work.

REFERENCES

- [1] Ministry of Agriculture & Farmers Welfare, Government of India, “State of Agriculture in India,” Annual Report, New Delhi, 2022.
- [2] N. Gondchawar and R. S. Kawitkar, “IoT based smart agriculture,” *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 5, no. 6, pp. 838–842, 2016.
- [3] A. Patil and B. Kumar, “Fertilizer recommendation system using machine learning for precision agriculture,” in *Proc. IEEE Int. Conf. Advances in Computing, Communications and Informatics (ICACCI)*, Jaipur, India, 2018, pp. 1421–1426.
- [4] S. Srivastava, A. Bhardwaj, and R. Singh, “Deep learning-based soil type classification from RGB images,” in *Proc. Int. Conf. on Computational Intelligence and Data Engineering (ICCIDE)*, Hyderabad, India, 2021, pp. 89–97.
- [5] A. Kamilaris, A. Kartakoullis, and F. X. Prenafeta-Boldú, “A review on the practice of big data analysis in agriculture,” *Computers and Electronics in Agriculture*, vol. 143, pp. 23–37, 2017.
- [6] P. Verma, R. Gupta, and N. Kaur, “Cloud-based real-time crop monitoring system using Firebase and IoT sensors,” in *Proc. IEEE Int. Conf. on Internet of Things: Smart Innovation and Uses (IoT-SIU)*, Ghaziabad, India, 2020, pp. 1–6.
- [7] Eclipse Foundation, “Mosquitto: An Open Source MQTT Broker,” 2023. [Online]. Available: <https://mosquitto.org>
- [8] OpenWeatherMap, “Current Weather and Forecast API,” 2024. [Online]. Available: <https://openweathermap.org/api>
- [9] Ministry of Electronics & IT, Government of India, “AGMARKNET Daily Mandi Price Dataset,” data.gov.in Resource ID: 9ef84268-d588-465a-a308-a864a43d0070, 2024.